

Recuperação Contínua — C#

Comparativo de algoritmos com estruturas condicionais e o uso do While

Estruturas abordadas: IF/Else · Switch · While

1. Quadro Comparativo

A tabela abaixo resume as principais diferenças entre as três estruturas estudadas.

Critério	IF / Else	Switch	While
Tipo	Condicional	Condicional	Repetição
Finalidade	Testar condições booleanas	Comparar variável a valores fixos	Repetir bloco enquanto condição for verdadeira
Quando usar	Condições complexas, intervalos	Valores discretos e fixos (menus)	Repetições com número variável
Avalia	Qualquer expressão booleana	Apenas igualdade (==)	Expressão booleana a cada iteração
Legibilidade	Pode ficar verboso com muitos else if	Mais limpo para muitos valores fixos	Simples para loops condicionais
Risco principal	Condição errada passa despercebida	Esquecer o break causa fall-through	Loop infinito se condição nunca for falsa
Alternativas	Switch, ternário	IF/Else	for, do-while, foreach

2. Estrutura IF / Else

O IF avalia uma expressão booleana e executa um bloco de código se ela for verdadeira. O ELSE captura todos os casos que não satisfazem a condição. Podem ser encadeados com ELSE IF para testar múltiplas condições em sequência.

Sintaxe básica

```
if (condicao)
{
    // executa se verdadeiro
}
```

```
else if (outra condicao)
{
// executa se a segunda for verdadeira
}
else
{
// executa se nenhuma for verdadeira
}
```

Exemplo — classificação de notas

```
int nota = 75;

if (nota >= 90)
{
Console.WriteLine("Conceito A");
}
else if (nota >= 70)
{
Console.WriteLine("Conceito B");
}
else if (nota >= 50)
{
Console.WriteLine("Conceito C");
}
else
{
Console.WriteLine("Reprovado");
}

// Saida: Conceito B
```

Operador ternário (forma compacta)

Quando há apenas dois caminhos possíveis, o operador ternário é uma alternativa mais concisa:

```
string resultado = (nota >= 50) ? "Aprovado" : "Reprovado";  
Console.WriteLine(resultado); // Aprovado
```

3. Estrutura Switch

O Switch compara uma variável a uma lista de valores fixos (cases). É ideal quando há muitas opções discretas para um mesmo dado, tornando o código mais organizado do que vários ELSE IF. Em C#, esquecer o break gera erro de compilação — uma proteção da linguagem.

Sintaxe básica

```
switch (variavel)  
{  
    case valor1:  
        // codigo  
        break;  
    case valor2:  
        // codigo  
        break;  
    default:  
        // nenhum case correspondeu  
        break;  
}
```

Exemplo — menu de opções

```
int opcao = 2;  
  
switch (opcao)  
{  
    case 1:  
        Console.WriteLine("Novo jogo");  
        break;  
    case 2:  
        Console.WriteLine("Carregar jogo");  
        break;
```

```
case 3:
    Console.WriteLine("Sair");
    break;
default:
    Console.WriteLine("Opcao invalida");
    break;
}
// Saida: Carregar jogo
```

Atenção — fall-through em C#

Diferente de C e Java, o C# exige o break em cada case. Se você omiti-lo, o compilador gera um erro, impedindo comportamentos inesperados.

4. Estrutura While

O While repete um bloco de código enquanto uma condição for verdadeira. A condição é testada ANTES de cada execução — por isso, se já for falsa no início, o bloco nunca chega a executar. É fundamental alterar a variável de controle dentro do loop para evitar um loop infinito.

Sintaxe básica

```
while (condicao)
{
    // executa enquanto condicao for true
    // IMPORTANTE: alterar a variavel da condicao aqui
}
```

Exemplo — contagem regressiva

```
int contador = 5;

while (contador > 0)
{
    Console.WriteLine("Contagem: " + contador);
    contador--; // decrementa para evitar loop infinito
}
```

```
Console.WriteLine("Fim!");  
// Saída: 5, 4, 3, 2, 1, Fim!
```

Exemplo — validação de entrada do usuário

```
int numero = 0;  
  
while (numero <= 0)  
{  
    Console.Write("Digite um numero positivo: ");  
    numero = int.Parse(Console.ReadLine());  
}  
  
Console.WriteLine("Voce digitou: " + numero);
```

Loop infinito — o que evitar

ERRADO — nunca para por falta de decremento/incremento:

```
int x = 1; while (x > 0) { Console.WriteLine(x); }
```

5. Exemplo Prático Completo

O programa abaixo simula um caixa eletrônico simplificado combinando as três estruturas estudadas em um único algoritmo coeso.

Estruturas usadas

While — mantém o menu ativo até o usuário escolher sair

Switch — decide qual ação executar com base na opção digitada

IF/Else — valida o valor do saque (negativo, insuficiente ou válido)

```
double saldo = 1000.00;
int opcao = 0;

while (opcao != 3) // repete ate o usuario sair
{
    Console.WriteLine("\n=== BANCO C# ===");
    Console.WriteLine("1 - Ver saldo");
    Console.WriteLine("2 - Sacar");
    Console.WriteLine("3 - Sair");
    Console.Write("Opcao: ");
    opcao = int.Parse(Console.ReadLine());

    switch (opcao)
    {
        case 1:
            Console.WriteLine("Saldo: R$ " + saldo);
            break;

        case 2:
            Console.Write("Valor a sacar: R$ ");
            double valor = double.Parse(Console.ReadLine());

            if (valor <= 0)
            {
                Console.WriteLine("Valor invalido.");
            }
            else if (valor > saldo)
            {
                Console.WriteLine("Saldo insuficiente.");
            }
            else
            {
                Console.WriteLine("Saque realizado com sucesso.");
                saldo -= valor;
            }
        }
    }
}
```

```
Console.WriteLine("Saldo insuficiente!");  
}  
else  
{  
    saldo -= valor;  
    Console.WriteLine("Saque realizado! Novo saldo: R$ " + saldo);  
}  
break;  
  
case 3:  
    Console.WriteLine("Ate logo!");  
    break;  
  
default:  
    Console.WriteLine("Opcao invalida.");  
    break;  
}  
}
```